

3.modelling

Joao Lopes

2026-03-25

Contents

1. BUILD A MODEL	3
1.1. LOOK AT DATA	3
1.2. FIT AND EVALUATE MODEL	4
1.3. USE MODEL TO PREDICT	4
2. BUILD A MODEL USING WORKFLOW	6
2.1. LOOK AT DATA	6
2.2. SPLIT DATA	7
2.3. CREATE MODEL	7
2.4. CREATE RECIPE	7
2.5. CREATE WORKFLOW	7
2.6. FIT A MODEL	8
2.7. EVALUATE MODEL	8
3. BUILD VARIOUS MODELS USING WORKFLOW	10
3.1. LOOK AT DATA	10
3.2. SPLIT DATA	10
3.3. CREATE WORKFLOW AND TUNE MODEL 1	11
3.4. CREATE WORKFLOW AND TUNE MODEL 2	12
3.5. FIT AND EVALUATE BEST MODEL	14

1. BUILD A MODEL

[from <https://www.tidymodels.org/start/models/>]

Problem: Predict the price of a diamond.

Formula: $\text{price} \sim \text{carat} + \text{cut}$

Model: `lm()`

```
library("skimr")           #function skim() for descriptive statistics
library("GGally")          #function ggpairs() for data visualization
library("hexbin")          #function geom_hex() for data visualization
library("dotwhisker")      #function dwplot() for model coefficients visualization
library("performance")    #function check_model() for linear model assumptions
library("tidymodels")      #collection of packages for modelling
library("tidyverse")       #collection of packages for data analysis

tidyverse_conflicts()
tidymodels_conflicts()
```

1.1. LOOK AT DATA

```
#loading data and making changes
set.seed(123)
lev_cut = c("Fair", "Good", "Very Good", "Premium", "Ideal")
diamond_data <- diamonds |>
  mutate(
    log_price = log(price),           #log transform
    log_carat = log(carat),          #log transform
    cut = fct(as.character(cut), levels = lev_cut), #drop Ordered information
    .keep = "used"
  ) |>
  slice_sample(n = 1000)

#inspect data
glimpse(diamond_data)

#descriptive statistics
skim(diamond_data)

#data visualization 1
diamond_data |>
  select(-c(carat, price)) |>
  ggpairs(progress = FALSE)

#data visualization 2
ggplot(diamond_data, aes(x = log_carat, y = log_price)) +
  geom_hex() +
  facet_wrap(~cut, ncol = 1)

#data visualization 3
ggplot(diamond_data, aes(x = log_carat, y = log_price, color = cut)) +
  geom_smooth(method = "lm", formula = "y ~ x")
```

1.2. FIT AND EVALUATE MODEL

```
#set functional form
?linear_reg

#set engine
?set_engine

#perform fit
?fit.model_spec()

#fit model
t1 <- Sys.time()
diamond_fit <- linear_reg() |>
  set_engine("lm") |>
  fit(formula = log_price ~ log_carat + cut, data = diamond_data)
Sys.time() - t1

#analyse model (classic regression table)
tidy(diamond_fit)
glance(diamond_fit)

#predict using same data used for the fitting
diamond_aug <- augment(diamond_fit, new_data = diamond_data)
diamond_aug

#evaluate model using R-square (over evaluation)
diamond_aug |>
  rsq(truth = log_price, estimate = .pred)

#evaluate model using RMSE (over evaluation)
diamond_aug |>
  yardstick::rmse(truth = log_price, estimate = .pred)

#plot results
tidy(diamond_fit) |>
  dwplot(vline = geom_vline(xintercept = 0, colour = "grey50", linetype = 2))

#check MLR assumptions
check_model(diamond_fit$fit)
```

1.3. USE MODEL TO PREDICT

```
#create new points
new_points <- expand_grid(
  log_carat = log(2),
  cut = c("Fair", "Very Good", "Ideal")
)
new_points

#get prediction in log scale
mean_pred <- predict(diamond_fit, new_data = new_points)
mean_pred
```

```

#get confidence interval in log scale
conf_int_pred <- predict(diamond_fit, new_data = new_points, type = "conf_int")
conf_int_pred

#plot points in natural scale
k <- exp(glance(diamond_fit)$sigma^2/2) #unbiased constant for log-transf
new_points |>
  bind_cols(mean_pred, conf_int_pred) |>
  ggplot(aes(x = cut)) +
  geom_point(aes(y = k*exp(.pred))) +
  geom_errorbar(
    aes(ymin = k*exp(.pred_lower), ymax = k*exp(.pred_upper)),
    width = .2
  ) +
  labs(x = "diamond cut", y = "diamond price")

```

2. BUILD A MODEL USING WORKFLOW

[from <https://www.tidymodels.org/start/recipes/>]

Problem: Predict if a flight is going to arrive late.

Formula: $\text{arr_delay} \sim \text{air_time} + \text{dep_time} + \text{distance} + \text{carrier} + \text{dest} + \text{origin} + \text{date_dow} + \text{date_month} + \text{date_USHoliday}$

Model: `glm(family = binomial(link = "logit"))`

```
library("nycflights13") #collection of datasets
library("skimr")        #function skim() for descriptive statistics
library("tidyverse")    #collection of packages for data analysis
library("GGally")       #function ggpairs() for data visualization
library("tidymodels")   #collection of packages for modelling
library("themis")       #function step_downsample() for tidymodels
```

2.1. LOOK AT DATA

```
#loading data and making changes
flight_data <-
  flights |>
  mutate(
    #create variable on arrival delay
    arr_delay = ifelse(arr_delay >= 30, "late", "on_time"),
    arr_delay = fct(arr_delay, levels = c("late", "on_time")),
    #create variable date
    date = lubridate::as_date(time_hour)
  ) |>
  #select the variables that are going to be used
  select(dep_time, flight, origin, dest, air_time, distance,
         carrier, date, arr_delay, time_hour) |>
  #exclude missing data
  drop_na() |>
  #create factor variables
  mutate(across(where(is.character), as.factor))

#looking at data
glimpse(flight_data)

#descriptive statistics
skim(flight_data)

#data visualization
set.seed(123)
flight_data |>
  slice_sample(n=1000) |>
  select(-c(
    time_hour, flight, #ID of the flights
    dest, carrier,     #too many levels
    dep_time, date     #time and data
  ))
) |>
select(arr_delay, where(is.factor), where(is.numeric)) |>
ggpairs(progress = FALSE)
```

2.2. SPLIT DATA

```
#create random data split
set.seed(222)
flight_split <- initial_split(flight_data, strata = arr_delay, prop = 3/4)

flight_train <- training(flight_split) #extract training data (n = 245510)
flight_test  <- testing(flight_split)  #extract testing data   (n = 81836)

#look for unbalance in binary dependent variable (training data)
flight_train |>
  count(arr_delay) |>
  mutate(prop = n/sum(n))

#look for unbalance in binary dependent variable (testing data)
flight_test |>
  count(arr_delay) |>
  mutate(prop = n/sum(n))
```

2.3. CREATE MODEL

```
#create model specification
lr_mod <-
  logistic_reg() |>
  set_engine("glm")
```

2.4. CREATE RECIPE

```
#create recipe
flight_rec <-
  #new recipe
  recipe(formula = arr_delay ~ ., data = flight_train) |>
  #add role to recipe
  update_role(flight, time_hour, new_role = "ID") |>
  #add date features
  step_date(date, features = c("dow", "month")) |>
  step_holiday(date, holidays = timeDate::listHolidays("US")) |>
  step_rm(date) |>
  #down sample to correct unbalance in dependent variable
  step_downsample(arr_delay, skip = TRUE, seed = 222) |>
  #convert categorical variables to dummies
  step_dummy(all_nominal_predictors()) |>
  #remove columns with zero variance
  step_zv(all_predictors()) |>
  #remove highly correlated variables
  step_corr(all_numeric_predictors(), threshold = 0.9) |>
  #normalize numeric data
  step_normalize(all_numeric_predictors())
```

2.5. CREATE WORKFLOW

```

#create workflow
flight_wflow <-
  #new workflow
  workflow() |>
  #add model
  add_model(lr_mod) |>
  #add recipe
  add_recipe(flight_rec)
flight_wflow

```

2.6. FIT A MODEL

```

#fit the model using training data
t1 <- Sys.time()
glm_fit <-
  flight_wflow |>
  fit(data = flight_train)
Sys.time() - t1

#analyse model (classic regression table)
tidy(glm_fit)
glance(glm_fit)

```

2.7. EVALUATE MODEL

```

#calculate predictions using testing data
flight_aug <-
  augment(glm_fit, new_data = flight_test)
flight_aug |>
  select(arr_delay, time_hour, flight, .pred_class, .pred_on_time) |>
  print(n = 20)

#evaluate model using confusion matrix
lr_cmat <- flight_aug |>
  conf_mat(truth = arr_delay, estimate = .pred_class)
lr_cmat

summary(lr_cmat)

#evaluate model using ROC curve
flight_aug |>
  roc_curve(truth = arr_delay, .pred_late) |>
  autoplot()

#evaluate model using ROC area
flight_aug |>
  roc_auc(truth = arr_delay, .pred_late)

#extract results
glm_fit |>
  tidy() |>
  filter(!term %in% c("(Intercept)")) |>

```



```
mutate(  
  importance = abs(statistic),  
  term = reorder(term, importance)  
) |>  
arrange(desc(abs(statistic))) |>  
slice_head(n = 20) |>  
ggplot(aes(x = importance, y = term)) +  
geom_col() +  
labs(x = "importance (statistics)")
```

3. BUILD VARIOUS MODELS USING WORKFLOW

[from <https://www.tidymodels.org/start/case-study/>]

Problem: Predict if a booking has children.

Formula: children ~ .

Model 1: glmnet(family = "binomial")

Model 2: ranger(classification = TRUE)

```
library("skimr")           #function skim() for descriptive statistics
library("glmnet")          #function glmnet() to fit GLMs with penalized mL
library("ranger")          #function ranger() to random forest models
library("vip")             #function vip() to plot "vars importance" for ML models
library("tidyverse")       #collection of packages for data analysis
library("GGally")          #function ggpairs() for data visualization
library("tidymodels")      #collection of packages for modelling
library("themis")          #function step_downsample() for tidymodels
```

3.1. LOOK AT DATA

```
#loading data and making changes
hotels <-
  read_csv("https://tidymodels.org/start/case-study/hotels.csv") |>
  mutate(across(where(is.character), as.factor))

#inspect data
glimpse(hotels)

#descriptive statistics
skim(hotels)

#data visualization
set.seed(123)
hotels |>
  slice_sample(n=1000) |>
  select(-c(
    required_car_parking_spaces, customer_type, #too unbalanced
    meal:assigned_room_type,                   #too many levels
    stays_in_weekend_nights, is_repeated_guest:days_in_waiting_list,
    total_of_special_requests,                  #too many 0s
    arrival_date                                #data
  )) |>
  select(children, where(is.factor), where(is.numeric)) |>
  ggpairs(progress = FALSE)
```

3.2. SPLIT DATA

```
#create random data split
set.seed(123)
hotel_split <- initial_split(hotels, strata = children, prop = 0.75)

hotel_other <- training(hotel_split) #extract training data (n = 37500)
```

```

hotel_test <- testing(hotel_split) #extract testing data (n = 12500)

#look for unbalance in binary dependent variable (training data)
hotel_other |>
  count(children) |>
  mutate(prop = n/sum(n))

#look for unbalance in binary dependent variable (testing data)
hotel_test |>
  count(children) |>
  mutate(prop = n/sum(n))

#create random split of training data (n_training = 30000, n_tuning = 7500)
set.seed(222)
hotel_val <- validation_split(hotel_other, strata = children, prop = 0.80)

```

3.3. CREATE WORKFLOW AND TUNE MODEL 1

```

#create model specification
?logistic_reg

lr_mod <-
  logistic_reg(
    penalty = tune(), #total amount of regularization
    mixture = 1       #choose ridge (0), lasso (1), or elastic models (0 to 1)
  ) |>
  set_engine("glmnet")

#create recipe
holidays <- c("AllSouls", "AshWednesday", "ChristmasEve", "Easter",
              "ChristmasDay", "GoodFriday", "NewYearsDay", "PalmSunday")

hotel_rec <-
  #new recipe
  recipe(children ~ ., data = hotel_other) |>
  #add date features
  step_date(arrival_date) |>
  step_holiday(arrival_date, holidays = holidays) |>
  step_rm(arrival_date) |>
  #down sample to correct unbalance in dependent variable
  step_downsample(children, skip = TRUE, seed = 222) |>
  #convert categorical variables to dummies
  step_dummy(all_nominal_predictors()) |>
  #remove columns with zero variance
  step_zv(all_predictors()) |>
  #remove highly correlated variables
  step_corr(all_numeric_predictors(), threshold = 0.9) |>
  #normalize numeric data
  step_normalize(all_predictors())

#create workflow
lr_wflow <-
  #new workflow
  workflow() |>

```

```

#add model
add_model(lr_mod) |>
#add recipe
add_recipe(hotel_rec)
lr_wflow

#fit 20 models using training data and evaluate using tuning data
t1 <- Sys.time()
lr_res <-
  lr_wflow |>
  tune_grid(
    resamples = hotel_val,
    grid = 20,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc)
  )
Sys.time() - t1

#plot tuning metrics
autoplot(lr_res) + expand_limits(y=c(0.5, 1))

#select best tuning parameter
lr_best <-
  lr_res |>
  select_best(metric = "roc_auc")
lr_best

#collect predictions using best tuned model
lr_pred <- lr_res |>
  collect_predictions(parameters = lr_best)

#evaluate best tuned model using confusion matrix
lr_pred |>
  mutate(
    .pred_class = factor(if_else(.pred_children < 0.5, "none", "children"))
  ) |>
  conf_mat(truth = children, estimate = .pred_class)

#evaluate best tuned model using ROC curve
lr_auc <-
  lr_pred |>
  roc_curve(children, .pred_children) |>
  mutate(model = "Logistic Regression")
autoplot(lr_auc)

#evaluate best tuned model using ROC area
lr_pred |>
  roc_auc(children, .pred_children)

```

3.4. CREATE WORKFLOW AND TUNE MODEL 2

```

#number of cores to parallelize computation
cores <- parallel::detectCores()

```

```

#create model specification
?rand_forest

rf_mod <-
  rand_forest(
    mtry = tune(), #number of predictors for each split
    min_n = tune(), #minimum number of data points in a node to stop the split
    trees = 1000    #number of trees contained in the ensemble
  ) |>
  set_engine("ranger", num.threads = cores) |>
  set_mode("classification")

#create workflow
rf_wflow <-
  #new workflow
  workflow() |>
  #add model
  add_model(rf_mod) |>
  #add recipe
  add_recipe(hotel_rec)
rf_wflow

#fit 10 models using training data and evaluate using tuning data
set.seed(222)
t1 <- Sys.time()
rf_res <-
  rf_wflow |>
  tune_grid(
    resamples = hotel_val,
    grid = 10,
    control = control_grid(save_pred = TRUE),
    metrics = metric_set(roc_auc)
  )
Sys.time() - t1

#plot tuning metrics
autoplot(rf_res) + expand_limits(y=c(0.5, 1))

#select best parameters
rf_best <-
  rf_res |>
  select_best(metric = "roc_auc")
rf_best

#collect predictions using best tuned model
rf_pred <-
  rf_res |>
  collect_predictions(parameters = rf_best)

#evaluate best tuned model using confusion matrix
rf_pred |>
  mutate(
    .pred_class = factor(if_else(.pred_children < 0.5, "none", "children"))
  ) |>

```

```

conf_mat(truth = children, estimate = .pred_class)

#evaluate best tuned model using ROC curve
rf_auc <-
  rf_pred |>
  roc_curve(children, .pred_children) |>
  mutate(model = "Random Forest")
autoplot(rf_auc)

#evaluate best tuned model using ROC area
rf_pred |> roc_auc(children, .pred_children)

```

3.5. FIT AND EVALUATE BEST MODEL

```

#compare best tuned models
bind_rows(rf_auc, lr_auc) |>
  ggplot(aes(x = 1 - specificity, y = sensitivity, col = model)) +
  geom_path(lwd = 1.0, alpha = 0.8) +
  geom_abline(lty = 3) +
  coord_equal() +
  theme_bw() +
  theme(legend.position = "bottom")

#build last model [random forest]
rf_mod_last <-
  rand_forest(
    mtry = rf_best$mtry,
    min_n = rf_best$min_n,
    trees = 1000
  ) |>
  set_engine("ranger", num.threads = cores, importance = "impurity") |>
  set_mode("classification")

#create last workflow
rf_wflow_last <-
  #random forest workflow
  rf_wflow |>
  #update with tuned model
  update_model(rf_mod_last)

#fit last model using training data and evaluate using testing data
set.seed(345)
t1 <- Sys.time()
rf_fit_last <-
  rf_wflow_last |>
  last_fit(split = hotel_split)
Sys.time() - t1

#collect predictions using last model
rf_pred_last <- rf_fit_last |>
  collect_predictions()

#evaluate model using confusion matrix

```

```
rf_pred_last |>
  conf_mat(truth = children, estimate = .pred_class)

#evaluate model using ROC curve
rf_pred_last |>
  roc_curve(children, .pred_children) |>
  autoplot()

#evaluate model using ROC area
rf_pred_last |>
  roc_auc(children, .pred_children)

#extract results
rf_fit_last |>
  extract_fit_parsnip() |>
  vip(num_features = 20, include_type = TRUE)
```